

DIGITAL FORENSICS

Laboratory Report

Lab 4B — Steganography Analysis

Steganography Investigation

Strings · Binwalk · LSB · Timestamp · EXIF

Report ID	DF-LAB-4B-2026
Course	Digital Forensics and Incident Handling
Topic	Steganography Analysis
Student	Dari Parruca, Anastasiia Bila
Date	11 June 2026
Classification	Academic — Not for Distribution

1. Introduction

This report documents the analysis of three steganography samples provided for the lab assignment:

- strings-stego.jpg
- binwalk-stego.jpg
- lsb-stego.jpg

The purpose of the lab was to identify and recover hidden information from image files using different steganography investigation techniques. The first sample was analyzed using the strings command, the second using binwalk, and the third using Least Significant Bit analysis with offline Python scripts.

The assignment also required the final report PDF timestamp to be falsified and verified using NTFS-related evidence, including \$STANDARD_INFORMATION (\$SI) and \$FILE_NAME (\$FN) attributes. A phone photo of the proof screen was taken and its EXIF metadata was removed before inclusion in the final submission ZIP.

2. Tools Used

strings	Extract printable strings from image files
grep	Search extracted output for keywords and flags
binwalk	Detect and extract embedded objects from files
unzip	Inspect ZIP archive contents
file	Identify file types
Python 3	Extract LSB data and create visual LSB output
Pillow / PIL	Process image pixels in Python
xdg-open	Open generated image files
PowerShell	Modify visible Windows file timestamps
fsutil file layout	Inspect NTFS file layout and show \$SI / \$FN attributes
exiftool	Inspect and remove EXIF metadata from the phone photo
zip	Create the final submission archive

3. Screenshot References

Screenshot ID	Description
SC-01	strings-stego.jpg analysis showing the recovered flag flag{5tr1ng5_f0r_l1f3}

SC-02	binwalk-stego.jpg scan and extraction showing embedded ZIP at offset 237364 / 0x39F34
SC-03	Extracted flag.txt and ZIP listing showing flag{b1nw@lk_15_l0v3ly}
SC-04	LSB extraction from lsb-stego.jpg showing flag{b1nw@lk_15_l0v3ly}
SC-05	Visual LSB bit-plane output and zoomed top-region result
SC-06	Timestamp falsification proof: PowerShell timestamp change and phone photo showing NTFS \$SI / \$FN attributes
SC-07	EXIF metadata removal proof using exiftool before and after stripping

4. Sample 1 — Analysis of strings-stego.jpg

4.1 Objective

The objective for the first sample was to use the strings command to locate hidden text inside strings-stego.jpg.

4.2 Method

The strings command was used to extract printable ASCII strings from the JPG file. The output was saved into a text file and searched for common keywords such as flag, ctf, hidden, secret, password, and key.

Commands used:

```
cd ~/Downloads/stego
strings strings-stego.jpg > strings_output.txt
cat strings_output.txt
grep -iE "flag|ctf|hidden|secret|password|key" strings_output.txt
```

The command strings strings-stego.jpg extracted readable character sequences from the image file. Since hidden text in simple steganography samples is often stored as printable data, the output was searched using grep.

4.3 Result

The keyword search revealed the following hidden flag:

RECOVERED FLAG — Sample 1

flag{5tr1ng5_f0r_11f3}

This confirmed that the first image contained hidden text that could be recovered using basic string extraction.

```

3Z[;
E3H$
h^)fP
k0o}!@
y~H' '
_h3J
>!jS@
(N8k
nWz|
*;}+
Klu{K
m|]q3G
flag{5tr1ng5_f0r_l1f3}
:}`h
cr251504@pc19:~/Downloads/stego$ grep -iE "flag|ctf|hidden|secret|password|key"
strings_output.txt
flag{5tr1ng5_f0r_l1f3}
cr251504@pc19:~/Downloads/stego$

```

Figure SC-01 — strings and grep output showing flag{5tr1ng5_f0r_l1f3}

4.4 Conclusion for Sample 1

The first sample was successfully analyzed using the strings command. The hidden text was recovered without requiring image decoding or object extraction. The recovered flag was flag{5tr1ng5_f0r_l1f3}.

5. Sample 2 — Analysis of binwalk-stego.jpg

5.1 Objective

The objective for the second sample was to locate and extract embedded objects from binwalk-stego.jpg. binwalk is suitable for identifying hidden file signatures and embedded archives.

5.2 Method

The image was scanned using binwalk:

```
binwalk binwalk-stego.jpg | tee binwalk_scan.txt
```

The scan returned the following relevant result:

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, JFIF standard 1.01
30	0x1E	TIFF image data, big-endian
418	0x1A2	Copyright string: "Copyright (c) 1998 Hewlett-Packard"
237364	0x39F34	Zip archive data, compressed: 23, uncompressed: 23, name: flag.txt
237515	0x39FCB	End of Zip archive

The important finding was the embedded ZIP archive at decimal offset 237364 (hexadecimal offset 0x39F34). The embedded object was then extracted:

```
binwalk -e binwalk-stego.jpg
```

```
# Inspect the extracted directory
```

```
ls -la _binwalk-stego.jpg.extracted
```

```
file _binwalk-stego.jpg.extracted/*
```

The extracted directory contained 39F34.zip and flag.txt. The file command confirmed that flag.txt was an ASCII text file.

```
cr251504@pc19:~/Downloads/stego$ binwalk binwalk-stego.jpg | tee binwalk_scan.txt
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, JFIF standard 1.01
30	0x1E	TIFF image data, big-endian, offset of first image directory: 8
418	0x1A2	Copyright string: "Copyright (c) 1998 Hewlett-Packard Company"
237364	0x39F34	Zip archive data, at least v1.0 to extract, compressed size: 23, uncompressed size: 23, name: flag.txt
237515	0x39FCB	End of Zip archive, footer length: 22

```
cr251504@pc19:~/Downloads/stego$ binwalk -e binwalk-stego.jpg
```

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, JFIF standard 1.01
30	0x1E	TIFF image data, big-endian, offset of first image directory: 8
418	0x1A2	Copyright string: "Copyright (c) 1998 Hewlett-Packard Company"
237364	0x39F34	Zip archive data, at least v1.0 to extract, compressed size: 23, uncompressed size: 23, name: flag.txt
237515	0x39FCB	End of Zip archive, footer length: 22

```
cr251504@pc19:~/Downloads/stego$ ls -la
ls -la _binwalk-stego.jpg.extracted
file _binwalk-stego.jpg.extracted/*
total 1040
drwxr-xr-x 3 cr251504 499 8 Mai 31 02:49 .
drwx----- 7 cr251504 499 25 Mai 31 02:45 ..
-rw-r--r-- 1 cr251504 499 579 Mai 31 02:49 binwalk_scan.txt
-rw-r--r-- 1 cr251504 499 237537 Mar 2 2020 binwalk-stego.jpg
drwxr-xr-x 2 cr251504 netlabs 4 Mai 31 02:49 _binwalk-stego.jpg.extracted
-rw-r--r-- 1 cr251504 499 574864 Mar 2 2020 lsb-stego.jpg
-rw-r--r-- 1 cr251504 499 10225 Mai 31 02:47 strings_output.txt
-rw-r--r-- 1 cr251504 499 163566 Mar 2 2020 strings-stego.jpg
total 14
drwxr-xr-x 2 cr251504 netlabs 4 Mai 31 02:49 .
drwxr-xr-x 3 cr251504 499 8 Mai 31 02:49 ..
-rw-r--r-- 1 cr251504 netlabs 173 Mai 31 02:49 39F34.zip
-rw-r--r-- 1 cr251504 netlabs 23 Apr 4 2019 flag.txt
_binwalk-stego.jpg.extracted/39F34.zip: Zip archive data, at least v1.0 to extract, compression method=store
_binwalk-stego.jpg.extracted/flag.txt: ASCII text, with no line terminators
```

Figure SC-02 — binwalk scan showing the embedded ZIP archive at 237364 / 0x39F34

5.3 Result

The extracted text file was read, and the ZIP contents were verified:

```
cat _binwalk-stego.jpg.extracted/flag.txt

# Verify ZIP contents
unzip -l _binwalk-stego.jpg.extracted/39F34.zip
unzip -p _binwalk-stego.jpg.extracted/39F34.zip flag.txt
```

The archive contained one file named flag.txt with a length of 23 bytes. The recovered flag was:

RECOVERED FLAG — Sample 2

flag{b1nw@lk_15_10v31y}

```
cr251504@pc19:~/Downloads/stego$ cat _binwalk-stego.jpg.extracted/flag.txt
flag{b1nw@lk_15_10v31y}cr251504@pc19:~/Downloads/stego$ unzip -l _binwalk-stego.jpg.extracted/39F34.zip
unzip -p _binwalk-stego.jpg.extracted/39F34.zip flag.txt
Archive: _binwalk-stego.jpg.extracted/39F34.zip
  Length      Date    Time    Name
  -----
    23      2019-04-04 21:59    flag.txt
  -----
    23
  1 file
```

Figure SC-03 — Extracted flag.txt and ZIP listing showing flag{b1nw@lk_15_10v31y}

5.4 Conclusion for Sample 2

The second sample contained an embedded ZIP archive inside the JPG file. binwalk identified it at decimal offset 237364 / hexadecimal 0x39F34. After extraction, flag.txt was recovered. The recovered flag was flag{b1nw@lk_15_10v31y}.

6. Sample 3 — Analysis of lsb-stego.jpg

6.1 Objective

The objective for the third sample was to investigate lsb-stego.jpg for hidden data embedded using Least Significant Bit steganography, and to demonstrate visual LSB un hiding by shifting the least significant bit to the most significant visible position.

6.2 LSB Extraction Method

An offline Python script was created to extract the least significant bit from each RGB color channel of the image. The extracted bits were grouped into bytes and converted into readable output.

Script: lsb_extract.py

```
from PIL import Image

img = Image.open("lsb-stego.jpg").convert("RGB")
bits = []

for r, g, b in img.getdata():
    bits.append(str(r & 1))
    bits.append(str(g & 1))
    bits.append(str(b & 1))

text = ""
for i in range(0, len(bits), 8):
    byte = bits[i:i+8]
    if len(byte) == 8:
        text += chr(int("".join(byte), 2))

print(text[:10000])
```

Execute and save output

```
python3 lsb_extract.py | tee lsb_output.txt
```

The output contained binary-like data including the PK signature, indicating that archive-like data was embedded inside the LSB bitstream.

6.3 Searching for the LSB-Embedded Flag

Because the extracted LSB output contained binary data, grep was used with the -a option to treat the output as text:

```
grep -a -o "flag{[^}]*}" lsb_output.txt
```

This confirmed that the LSB-embedded flag was successfully recovered from lsb-stego.jpg:

RECOVERED FLAG — Sample 3 (LSB)

flag{b1nw@1k_15_10v31y}

```
tr251504@pc19:~/Downloads/stego$ strings -a lsb_output.txt | grep -iE "flag|ctf|hidden|secret|key"
flag.txtflag{b1nw@1k_15_10v31y}PK
flag.txt
tr251504@pc19:~/Downloads/stego$ grep -a -o "flag{[^}]*}" lsb_output.txt
flag{b1nw@1k_15_10v31y}
```

Figure SC-04 — LSB extraction from lsb-stego.jpg showing flag{b1nw@1k_15_10v31y}

6.4 Binary Payload Investigation

A second Python script was used to save the reconstructed bitstream as a binary file and probe for ZIP structure:

Script: lsb_extract_binary.py

```
from PIL import Image
import re

img = Image.open("lsb-stego.jpg").convert("RGB")
bits = []
for r, g, b in img.getdata():
    bits.append(r & 1)
    bits.append(g & 1)
    bits.append(b & 1)

payload = bytearray()
for i in range(0, len(bits), 8):
    byte_bits = bits[i:i+8]
    if len(byte_bits) == 8:
        value = 0
        for bit in byte_bits:
            value = (value << 1) | bit
        payload.append(value)

with open("lsb_payload.bin", "wb") as f:
    f.write(payload)

match = re.search(b"PK\x03\x04", payload)
if match:
    start = match.start()
    with open("lsb_payload.zip", "wb") as f:
        f.write(payload[start:])
    print("ZIP header found at byte offset:", start)
else:
    print("No ZIP header found")

python3 lsb_extract_binary.py

# Verify payload type
file lsb_payload.bin
unzip -l lsb_payload.zip
```

The script confirmed ZIP header found at byte offset 0. The file command identified the payload as ZIP archive data. However, unzip could not fully extract the archive because the end-of-central-directory signature was missing, indicating the recovered payload was truncated. Despite this, the hidden flag was successfully recovered directly from the LSB output using grep.

6.5 Visual LSB Unhiding

Visual LSB unhiding was performed by shifting the LSB of each RGB channel into a visible position. LSB value 1 → channel 255; LSB value 0 → channel 0.

Script: lsb_visual.py

```
from PIL import Image

img = Image.open("lsb-stego.jpg").convert("RGB")
out = Image.new("RGB", img.size)
pixels = []

for r, g, b in img.getdata():
    nr = (r & 1) * 255
```

```

    ng = (g & 1) * 255
    nb = (b & 1) * 255
    pixels.append((nr, ng, nb))

out.putdata(pixels)
out.save("lsb_visual_unhidden.png")

```

The first output appeared mostly black with visible colored/noisy pixels near the top, as expected — the hidden payload was archive/text data, not an image. To make the pattern clearer, the top portion was cropped and enlarged:

Script: lsb_visual_zoom.py

```

from PIL import Image

img = Image.open("lsb_visual_unhidden.png").convert("RGB")
crop = img.crop((0, 0, img.width, 80))
zoom = crop.resize((img.width, 800), Image.NEAREST)
zoom.save("lsb_visual_top_zoom.png")

python3 lsb_visual.py
python3 lsb_visual_zoom.py
xdg-open lsb_visual_top_zoom.png

```

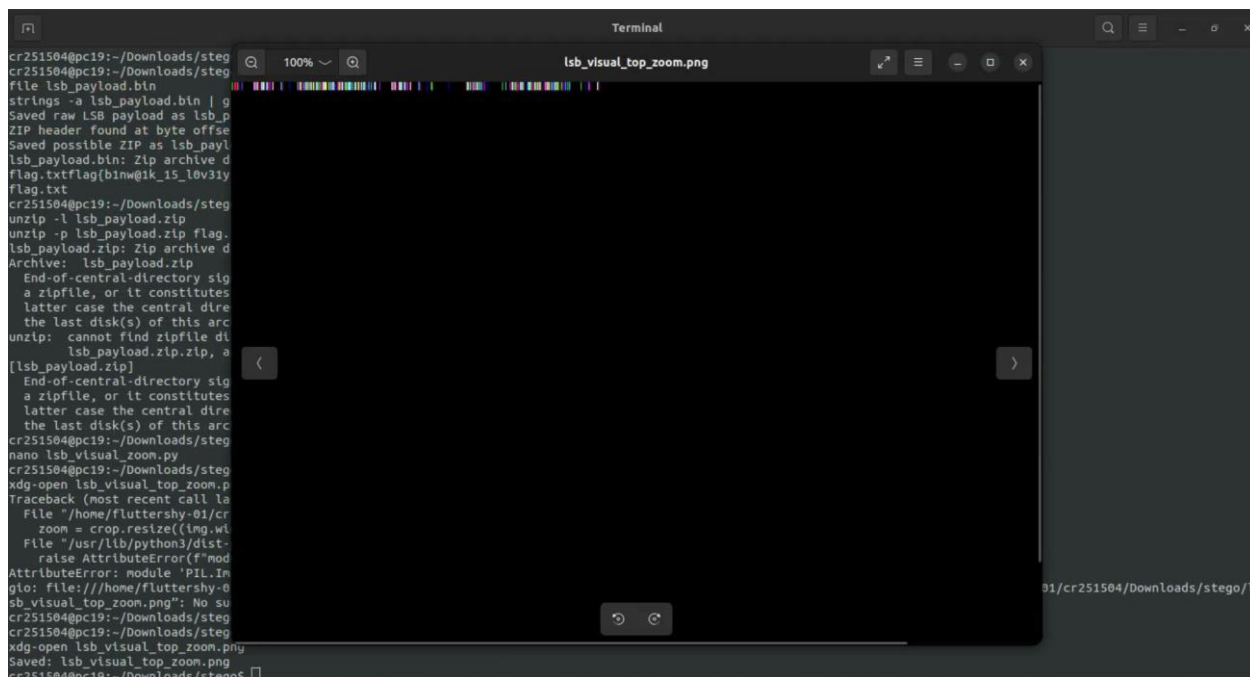


Figure SC-05 — Visual LSB bit-plane output and zoomed top-region result

6.6 Conclusion for Sample 3

The file lsb-stego.jpg contained data hidden in the least significant bits of its RGB pixel values. By extracting these bits and reconstructing them into bytes, the embedded flag was recovered as flag{b1nw@lk_15_l0v3ly}. The recovered data also showed ZIP-like content beginning with the PK signature. Visual LSB un hiding demonstrated that the LSB bit-plane contained structured data in the upper rows of the image.

7. Timestamp Falsification of the Report PDF

7.1 Objective

The assignment required the final report PDF timestamp to be falsified with NTFS evidence collected showing the \$STANDARD_INFORMATION (\$SI) and \$FILE_NAME (\$FN) attributes. This was performed on a Windows NTFS file system.

7.2 Method

The final report was exported as Steganography Lab Report.pdf and copied to C:\StegoLab\. PowerShell was used to modify the visible file timestamps:

Commands used (PowerShell):

```
$file = "C:\StegoLab\Steganography Lab Report.pdf"
$date = "2024-01-15 10:20:00"

(Get-Item $file).CreationTime = $date
(Get-Item $file).LastWriteTime = $date
(Get-Item $file).LastAccessTime = $date

Get-Item $file | Select-Object Name, CreationTime, LastWriteTime, LastAccessTime
```

After changing the timestamps, the NTFS file layout was inspected using fsutil:

```
fsutil file layout "C:\StegoLab\Steganography Lab Report.pdf" > C:\StegoLab\mft_proof.txt
notepad C:\StegoLab\mft_proof.txt
```

7.3 Result

The PowerShell output showed timestamps changed to 1/15/2024 10:20:00 AM. The fsutil file layout output confirmed the falsified values and the NTFS attributes:

Creation Time	15/01/2024 09:20:00 (falsified)
Last Access Time	15/01/2024 09:20:00 (falsified)
Last Write Time	15/01/2024 09:20:00 (falsified)
Change Time	31/05/2026 01:34:11 (reflects actual metadata update)
\$SI Attribute	Present in NTFS file record
\$FN Attribute	Present in NTFS file record

The following screenshots document the timestamp falsification evidence:

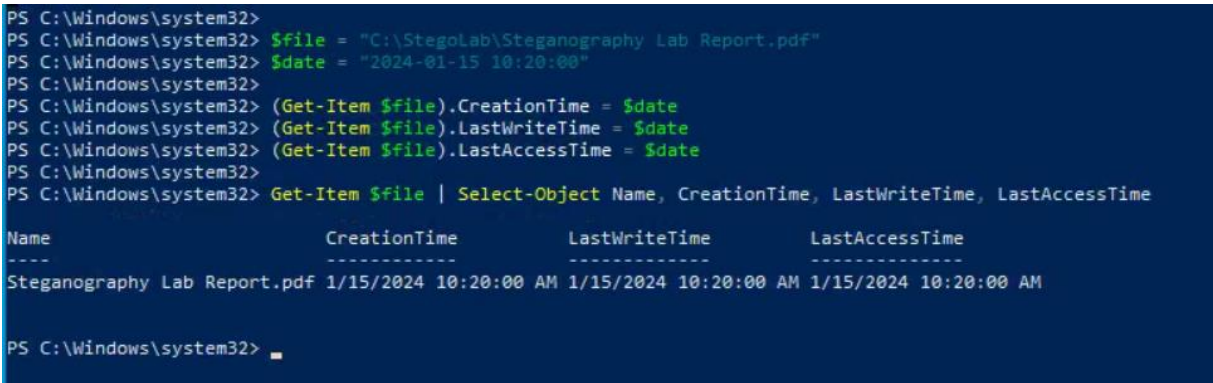


Figure SC-06a — PowerShell timestamp change confirming falsified file times

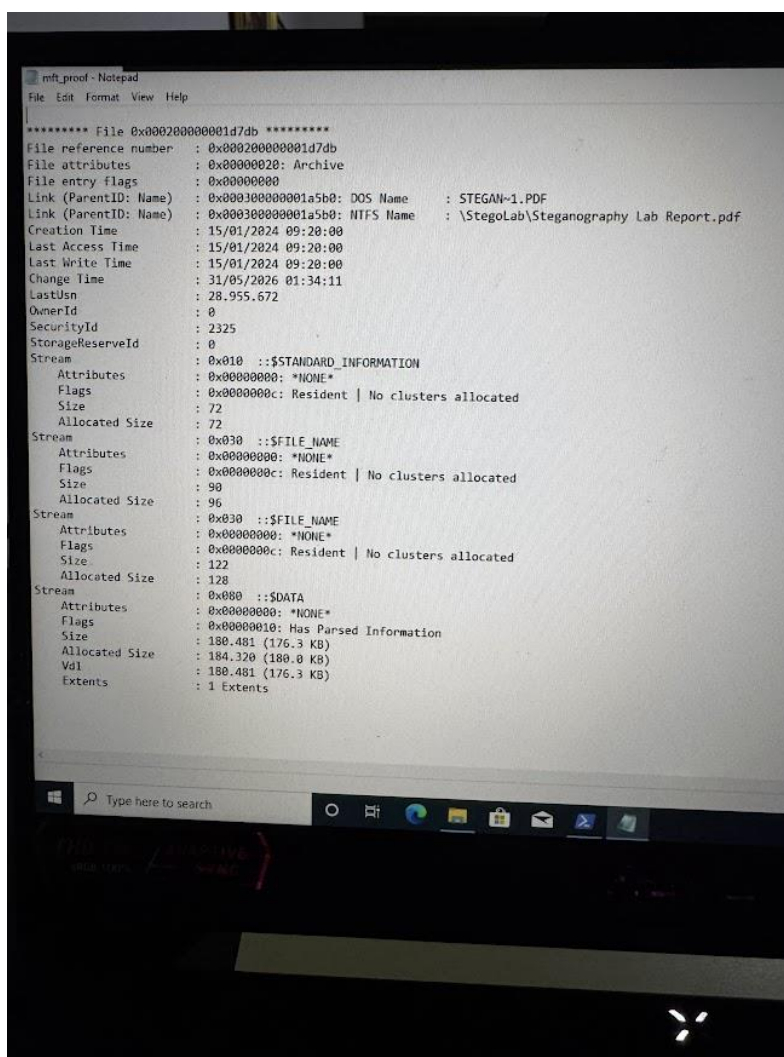


Figure SC-06b — Phone photo showing timestamp falsification proof and NTFS \$SI / \$FN attributes

7.4 Conclusion

The timestamp falsification requirement was completed on the final report PDF. The visible Windows timestamps were changed using PowerShell, and NTFS evidence was collected using fsutil file layout showing the \$STANDARD_INFORMATION and \$FILE_NAME attributes.

8. EXIF Metadata Removal from the Phone Photo

8.1 Objective

The assignment required a phone photo showing proof of the timestamp falsification and tool output. The EXIF metadata had to be stripped from the photo before it was added to the final submission ZIP file.

8.2 Method

The phone photo was transferred to the Linux environment as a HEIF/HEIC file named `cleaned_phone_photo.heif`. Metadata was first inspected, then removed:

```
# Inspect EXIF before removal
```

```
exiftool cleaned_phone_photo.heif | tee exif_before.txt

# Strip all metadata
exiftool -all= -overwrite_original cleaned_phone_photo.heif

# Verify after removal
exiftool cleaned_phone_photo.heif | tee exif_after.txt
```

Before removal, the file contained device and camera metadata including Make (Apple), Camera Model Name (iPhone 15 Pro Max), Host Computer, Date/Time Original, Create Date, lens information, and exposure information.

8.3 Result

The second exiftool check confirmed that personal camera and device EXIF fields were removed. Remaining fields (File Name, File Size, File Type, MIME Type, Image Width, Image Height, HEIC format information) are normal container-level fields that describe the file format rather than personal EXIF metadata.

```
Image Size           : 4032x3024
Megapixels           : 12.2
Scale Factor To 35 mm Equivalent: 10.8
Shutter Speed        : 1/50
Create Date          : 2026:05:31 03:36:18.713+02:00
Date/Time Original   : 2026:05:31 03:36:18.713+02:00
Modify Date          : 2026:05:31 03:36:18+02:00
Circle Of Confusion  : 0.003 mm
Field Of View        : 73.7 deg
Focal Length         : 2.2 mm (35 mm equivalent: 24.0 mm)
Hyperfocal Distance  : 0.81 m
Light Value          : 6.9
Lens ID              : iPhone 15 Pro Max back triple camera 2.22mm f/2.2
cr251504@pc19:~/Downloads$ exiftool -all= -overwrite_original cleaned_phone_photo.heif
Warning: ICC Profile deleted. Image colors may be affected - cleaned_phone_photo.heif
1 image files updated
cr251504@pc19:~/Downloads$ exiftool cleaned_phone_photo.heif | tee exif_after.txt
ExifTool Version Number : 12.40
File Name               : cleaned_phone_photo.heif
Directory               : .
File Size               : 1530 KiB
File Modification Date/Time : 2026:05:31 03:47:17+02:00
File Access Date/Time    : 2026:05:31 03:47:17+02:00
File Inode Change Date/Time : 2026:05:31 03:47:17+02:00
File Permissions         : -rw-r--r--
File Type               : HEIC
File Type Extension      : heic
MIME Type               : image/heic
Major Brand              : High Efficiency Image Format HEVC still image (.HEIC)
Minor Version           : 0.0.0
Compatible Brands        : mlf1, MlHB, MlHE, MlPr, mlaF, heic, tnap
Handler Type            : Picture
Primary Item Reference   : 49
Meta Image Size         : 4032x3024 0 515 2016 1512 0 0 64 0 0 15 16960 34 4198 15 16960 65535 63762 15 16960 34 4198 15 16960 12 27533 15 16960 0 10 15 16960 0
10 15 16960
XMP Toolkit              : XMP Core 6.0.0
HDR Gain Map Version     : 131072
HDR Gain Map Headroom   : 4.699222
Image Width             : 4032
Image Height            : 3024
Image Spatial Extent     : 4032x3024
Auxiliary Image Type     : urn:com:apple:photo:2020:aux:hdrgainmap
Color Representation     : nclx 2 2 2
Rotation                : 0
Image Pixel Depth        : 10 10 10
HEVC Configuration Version : 1
General Profile Space    : Conforming
General Tier Flag        : Main Tier
General Profile IDC      : Main Still Picture
Gen Profile Compatibility Flags : Main Still Picture, Main 10, Main
Constraint Indicator Flags : 176 0 0 0 0
General Level IDC        : 63 (level 2.1)
Min Spatial Segmentation IDC : 0
Parallelism Type         : 0
Chroma Format            : 4:2:0
Bit Depth Luma           : 8
Bit Depth Chroma         : 8
Average Frame Rate       : 0
```

Figure SC-07 — EXIF metadata removal proof: before metadata, stripping command, and after-removal verification

8.4 Conclusion

The phone photo metadata was successfully removed using exiftool. The cleaned file `cleaned_phone_photo.heif` was prepared for inclusion in the final submission ZIP.

9. Final Submission ZIP

The final submission ZIP was prepared after completing all lab tasks: steganography analysis, timestamp falsification, and EXIF metadata removal.

Files included:

- Steganography Lab Report.pdf
- cleaned_phone_photo.heif

ZIP creation and verification:

```
cd ~/Downloads
zip -r stego_submission.zip "Steganography Lab Report.pdf" cleaned_phone_photo.heif

# Verify contents
unzip -l stego_submission.zip
```

10. Summary of Findings

Sample	Technique Used	Main Finding
strings-stego.jpg	strings and grep	Hidden printable flag recovered
binwalk-stego.jpg	binwalk extraction	Embedded ZIP archive recovered
lsb-stego.jpg	LSB extraction with Python	LSB-embedded flag recovered
lsb-stego.jpg	Visual LSB shifting	LSB bit-plane visualized as colored bars
Report PDF	PowerShell and fsutil layout	Timestamp falsification proof collected
Phone photo	exiftool	EXIF metadata removed

Recovered flags:

strings-stego.jpg	flag{5tr1ng5_f0r_l1f3}
binwalk-stego.jpg	flag{b1nw@lk_15_l0v3ly}
lsb-stego.jpg	flag{b1nw@lk_15_l0v3ly}

11. Final Conclusion

All three steganography samples were successfully investigated. The first sample contained hidden printable text recovered using strings. The second sample contained an embedded ZIP archive detected and extracted using binwalk. The third sample contained data hidden using Least Significant Bit steganography — the LSB data was extracted using offline Python scripts and the hidden flag was recovered from the reconstructed bitstream.

Visual LSB unhiding was demonstrated by shifting the least significant bits into visible positions. The generated output showed colored/noisy bars rather than a recognizable image because the embedded payload was archive/text data.

The final report PDF timestamp was falsified using PowerShell, and NTFS evidence was collected using fsutil file layout, which showed the \$STANDARD_INFORMATION and \$FILE_NAME attributes. A phone photo was taken as required, and its EXIF metadata was stripped using exiftool before submission.